Secure Cloud Application Architecture with Automated Threat Modeling and Risk Mitigation

Devansh Gupta (23114024)
Daidipya Mathur (23114022)

Prof. Awanish Pandey
Cloud Security

March 12, 2026

Contents

1 Introduction and Objectives **2**
1.1 Project Overview 2

2 Secure Cloud Architecture Design **2**
2.1 N-Tier Trust Boundaries 2
2.2 High-Performance Backend Implementation 2

3 Authentication and Authorization **3**
3.1 Cryptographic Credential Management 3
3.2 Stateless Role-Based Access Control (RBAC) 3

4 Automated Threat Modeling **3**
4.1 Spoofing 3
4.2 Tampering 3
4.3 Repudiation 3
4.4 Information Disclosure 3
4.5 Denial of Service 4
4.6 Elevation of Privilege 4

5 Automated Risk Mitigation **4**
5.1 The Automated Monitor Daemon 4
5.2 Dynamic IP Ban Execution 4

6 Results and Incident Aftermath **4**
6.1 Brute Force Mitigation Protocol 4
6.2 JWT Forgery Rejection 5
6.3 Denial of Service Suppression 6

7 Conclusion **7**

1 Introduction and Objectives

1.1 Project Overview

- This project builds a reliable cloud architecture. It pairs a fast C++ backend with strict edge network filtering to keep the system secure without slowing it down.
- Our main goals were to set up **Role-Based Access Control (RBAC)**, perform automated threat modeling, and create an active defense system that stops bad traffic on the fly.

2 Secure Cloud Architecture Design

2.1 N-Tier Trust Boundaries

- The **Nginx API Gateway** serves as our front door. It hides the internal C++ server and SQLite database from the public internet by handling all incoming connections at port 8081.
- Internal communication is carefully controlled. The internal ban endpoint automatically drops any requests that don't come from a verified local source, preventing bypass attacks.

2.2 High-Performance Backend Implementation

- We wrote the core application in C++ to take advantage of its low latency and tight memory control, which keeps the system stable even under heavy traffic.
- Important shared resources, like the log files and active IP blocklist, are protected using **mutex locks**. This prevents the server from crashing due to race conditions during simultaneous attacks.

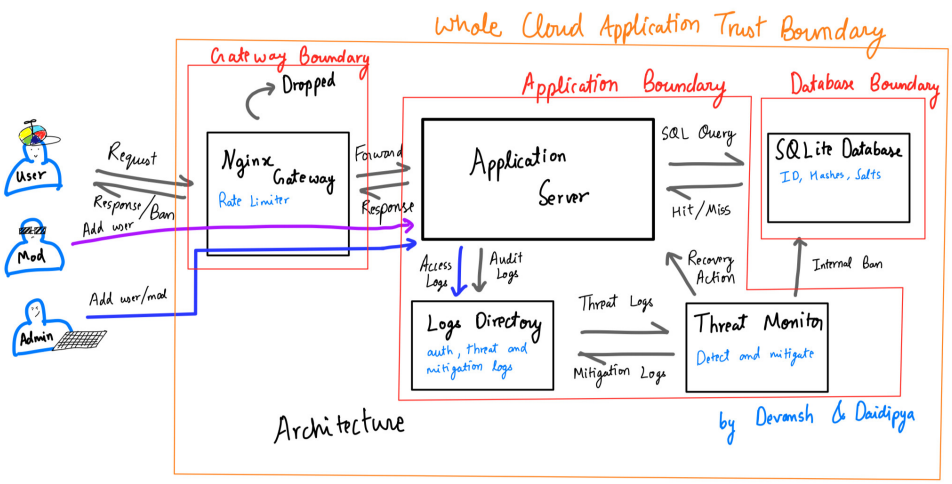


Figure 1: N-Tier Secure Cloud Architecture Data Flow

3 Authentication and Authorization

3.1 Cryptographic Credential Management

- We protect user passwords stored in the database by combining them with random salts generated by OpenSSL and hashing them with **SHA-256**.
- To prevent **SQL Injection**, the database relies entirely on prepared parameterized statements. This ensures that user inputs can never be executed as malicious code.

3.2 Stateless Role-Based Access Control (RBAC)

- When a user logs in successfully, the server issues a securely signed **JSON Web Token (JWT)**. This token records whether the user is an admin, moderator, or standard user.
- Future requests must include this token. The server can cryptographically verify the user's permissions directly from the token, saving the time and effort of constantly querying the database.

4 Automated Threat Modeling

4.1 Spoofing

To prevent attackers from pretending to be legitimate users, we enforce strict, short-lived session token expiration times. Every incoming request to a protected route must pass cryptographic signature validation, ensuring the user is exactly who they claim to be.

4.2 Tampering

We systematically prevent data tampering by hashing our JSON Web Tokens with the **HMAC-SHA256** algorithm. If an attacker alters even a single character of their role or user data, the mathematical signature breaks, and the server immediately rejects it.

4.3 Repudiation

To stop users from denying their actions, the system features an immutable, thread-safe logging infrastructure. Every successful login, failed attempt, and automated ban is permanently recorded with an exact timestamp and IP address to guarantee a reliable audit trail.

4.4 Information Disclosure

We protect sensitive system information by having the Nginx gateway strip away backend server identity headers. Additionally, the C++ framework is designed to catch internal code exceptions safely, returning only generic error messages to users instead of revealing stack traces.

4.5 Denial of Service

Volumetric floods and DoS attacks are neutralized at the network perimeter. We configured Nginx with strict **rate-limiting** zones. This allows it to absorb and drop excessive request spikes before they can ever reach and exhaust the C++ processing threads.

4.6 Elevation of Privilege

We explicitly block users from accessing administrative functions by using strict programmatic middleware. Before executing any sensitive internal operations, the server checks the securely signed JWT to verify that the user's role claim is genuinely set to administrator.

5 Automated Risk Mitigation

5.1 The Automated Monitor Daemon

- We built a separate C++ background program that acts as our security monitor. It watches the authentication logs in real-time without interfering with the main web server.
- Because it runs independently, this **out-of-band** setup ensures that even if the web server is overwhelmed with traffic, the monitor still has the CPU resources it needs to analyze threats.

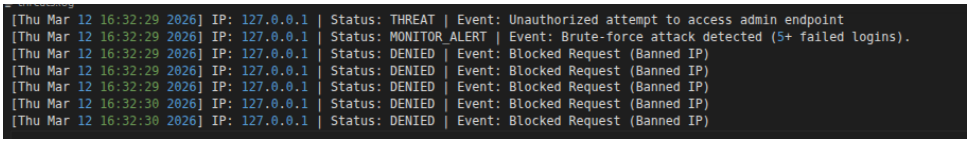
5.2 Dynamic IP Ban Execution

- If the monitor sees five failed login attempts from the same IP within a minute, it automatically sends a ban request to the application server.
- The server then adds that IP to its **in-memory blocklist** and drops all future connections from them. This entire defense process happens without any human intervention.

6 Results and Incident Aftermath

6.1 Brute Force Mitigation Protocol

- When we intentionally spammed the server with wrong passwords, the monitor correctly identified the attack and blocked us, returning a **403 Forbidden** status.
- The system then logged the exact details of the ban into a dedicated mitigation log file, keeping a clean and reliable audit trail for administrators.



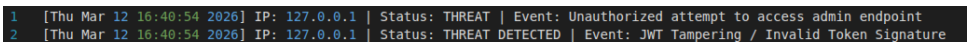
```
[Thu Mar 12 16:32:29 2026] IP: 127.0.0.1 | Status: THREAT | Event: Unauthorized attempt to access admin endpoint
[Thu Mar 12 16:32:29 2026] IP: 127.0.0.1 | Status: MONITOR_ALERT | Event: Brute-force attack detected (5+ failed logins).
[Thu Mar 12 16:32:29 2026] IP: 127.0.0.1 | Status: DENIED | Event: Blocked Request (Banned IP)
[Thu Mar 12 16:32:29 2026] IP: 127.0.0.1 | Status: DENIED | Event: Blocked Request (Banned IP)
[Thu Mar 12 16:32:29 2026] IP: 127.0.0.1 | Status: DENIED | Event: Blocked Request (Banned IP)
[Thu Mar 12 16:32:30 2026] IP: 127.0.0.1 | Status: DENIED | Event: Blocked Request (Banned IP)
[Thu Mar 12 16:32:30 2026] IP: 127.0.0.1 | Status: DENIED | Event: Blocked Request (Banned IP)
```

Figure 2: Log output displaying the automated detection and mitigation of a brute-force attack.

6.2 JWT Forgery Rejection

- We tried to access protected data using a modified JWT. The server instantly recognized that the signature didn't match and rejected the request.
- Not only did it stop the unauthorized access, but it also wrote a specific warning to the threat log, recording the exact IP address that attempted the forgery.

```
[Thu Mar 12 17:08:13 2026] IP: 127.0.0.1 | Status: THREAT_DETECTED |
Event: JWT Tampering / Invalid Token Signature
[Thu Mar 12 17:08:13 2026] IP: 127.0.0.1 | Status: THREAT_DETECTED |
Event: JWT Tampering / Invalid Token Signature
[Thu Mar 12 17:08:13 2026] IP: 127.0.0.1 | Status: THREAT_DETECTED |
Event: JWT Tampering / Invalid Token Signature
```



```
[Thu Mar 12 16:40:54 2026] IP: 127.0.0.1 | Status: THREAT | Event: Unauthorized attempt to access admin endpoint
[Thu Mar 12 16:40:54 2026] IP: 127.0.0.1 | Status: THREAT DETECTED | Event: JWT Tampering / Invalid Token Signature
```

Figure 3: Terminal evidence of the system intercepting and blocking a manipulated JSON Web Token.

6.3 Denial of Service Suppression

- We ran a script to bombard the external gateway with requests. This successfully triggered the rate-limiting rules we set up in Nginx.
- Nginx absorbed the brunt of the attack and returned a **503 Service Unavailable** error to the attacker, completely shielding the fragile C++ backend from the flood.

```
2026/03/12 16:40:51 [error] 61611#61611: *49 limiting requests, excess: 20.790 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *50 limiting requests, excess: 20.720 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *51 limiting requests, excess: 20.630 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *52 limiting requests, excess: 20.520 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *53 limiting requests, excess: 20.440 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *54 limiting requests, excess: 20.340 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *55 limiting requests, excess: 20.220 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *56 limiting requests, excess: 20.130 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *57 limiting requests, excess: 20.060 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *60 limiting requests, excess: 20.880 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *61 limiting requests, excess: 20.780 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *62 limiting requests, excess: 20.660 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *63 limiting requests, excess: 20.590 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *64 limiting requests, excess: 20.500 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61611#61611: *65 limiting requests, excess: 20.340 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *66 limiting requests, excess: 20.220 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *67 limiting requests, excess: 20.050 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *68 limiting requests, excess: 20.040 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *71 limiting requests, excess: 20.690 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *72 limiting requests, excess: 20.580 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *73 limiting requests, excess: 20.400 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *74 limiting requests, excess: 20.230 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *75 limiting requests, excess: 20.110 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *78 limiting requests, excess: 20.860 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *79 limiting requests, excess: 20.690 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *80 limiting requests, excess: 20.500 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *81 limiting requests, excess: 20.460 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *82 limiting requests, excess: 20.290 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *83 limiting requests, excess: 20.110 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61615#61615: *84 limiting requests, excess: 20.020 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *87 limiting requests, excess: 20.770 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *88 limiting requests, excess: 20.610 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *89 limiting requests, excess: 20.520 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *90 limiting requests, excess: 20.390 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *91 limiting requests, excess: 20.210 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *92 limiting requests, excess: 20.100 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:51 [error] 61616#61616: *93 limiting requests, excess: 20.060 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:52 [error] 61616#61616: *96 limiting requests, excess: 20.850 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:52 [error] 61616#61616: *97 limiting requests, excess: 20.720 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:52 [error] 61616#61616: *98 limiting requests, excess: 20.570 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
2026/03/12 16:40:52 [error] 61616#61616: *99 limiting requests, excess: 20.500 by zone "api_limit", client: 127.0.0.1, server: localhost, request: "GET /whoami
```

Figure 4: Execution of rate-limiting policies suppressing a simulated denial-of-service flood.

7 Conclusion

The final architecture successfully meets the requirements for a secure cloud application. By placing a fast C++ backend behind a strict Nginx proxy, we achieved solid **Defense in Depth**. Pairing stateless JWTs with an automated security monitor keeps the system fast and responsive while actively neutralizing threats in real-time.